MercadoTico 360: Diseño e implementación de una arquitectura NoSQL orientada a documentos en MongoDB Atlas para la gestión masiva y escalable de comercio electrónico

Adrián Quesada Jiménez, Camila Alpízar Alfaro, Emilio Calderón Jiménez, Gustavo Jiménez Hidalgo, Justin Ariel Zhu Fan, Salma Capín Romero

Escuela de estadística, Universidad de Costa Rica
San José, Costa Rica
adrian.quesadajimenez@ucr.ac.cr
camila.alpizar@ucr.ac.cr
gabriel.calderonjimenez@ucr.ac.cr
gustavo.jimenezhidalgo@ucr.ac.cr
justin.zhu@ucr.ac.cr
capinsalma@gmail.com,

***Resumen* – El crecimiento del comercio electrónico exige arquitecturas de almacenamiento capaces de gestionar datos masivos, heterogéneos y de alta velocidad. Los sistemas de bases de datos relacionales tradicionales presentan severas limitaciones al soportar catálogos diversificados y evolutivos, generando rigidez estructural, exceso de campos nulos y degradación del rendimiento transaccional. En este trabajo se diseña e implementa una arquitectura NoSQL orientada a documentos en MongoDB Atlas para la plataforma de comercio electrónico MercadoTico 360. El modelo propuesto consolida un catálogo heterogéneo (alimentos, ropa, tecnología, artesanías y servicios) dentro de una única colección mediante un esquema flexible, empleando además el patrón de documentos embebidos en los pedidos para garantizar la inmutabilidad e integridad histórica de las transacciones. La evaluación experimental sobre un conjunto de datos sintéticos de 85 500 documentos demostró que la estrategia de indexación reduce los tiempos de respuesta en más del 85 % en consultas clave, acelerando la recuperación de pedidos de ~115–235 ms a 0.5 ms. Asimismo, el sistema sostuvo un procesamiento de hasta 621 solicitudes por segundo bajo cargas concurrentes. Se concluye que la combinación de esquemas orientados a documentos con una adecuada indexación constituye una solución altamente eficiente, flexible y escalable para el comercio electrónico contemporáneo.**

***Palabras clave* – Base de datos NoSQL, MongoDB Atlas, modelo orientado a documentos, catálogo heterogéneo, comercio electrónico, inmutabilidad transaccional, escalabilidad, indexación.**

I. INTRODUCCIÓN

A. Planteamiento del problema

En el entorno digital contemporáneo, las plataformas de comercio electrónico se enfrentan al desafío de gestionar volúmenes masivos de datos caracterizados por su alta velocidad de generación y su heterogeneidad estructural. La tecnología intensiva en datos ha tenido una rápida expansión, lo que ha llevado a los sistemas tradicionales de bases de datos relacionales al límite, con lo cual se ha impulsado la adopción de bases de datos NoSQL como MongoDB [1].

Los modelos de bases de datos relacionales tradicionales, basados en esquemas rígidos de tablas y columnas fuertemente normalizadas, presentan severas limitaciones cuando intentan soportar catálogos de productos altamente diversificados y evolutivos en tiempo real. Para el caso de negocio de MercadoTico 360, esta problemática se manifiesta en tres puntos operacionales críticas:

1. **Inflexibilidad del catálogo:** La coexistencia de categorías diversas, como alimentos, ropa, tecnología, artesanías y servicios, plantea desafíos para el diseño de un catálogo unificado. Bajo un modelo relacional, las diferencias entre los atributos de cada categoría

requerirían del uso de tablas adicionales y operaciones de unión (*JOINs*), o bien conducir a diseños con numerosos campos opcionales. Es en este punto, las bases de datos NoSQL orientadas a documentos ofrecen flexibilidad para representar registros con estructuras diferentes y facilitar la incorporación de nuevas categorías o atributos sin exigir que todos los documentos compartan exactamente el mismo esquema [2]

2. **Degradación del rendimiento y pérdida de integridad histórica:** El procesamiento de pedidos transaccionales que dependen de catálogos dinámicos, pone en riesgo la inmutabilidad del historial de compras si un producto cambia de precio o atributos en el futuro, además de ralentizar las consultas debido a la fragmentación de la información de las líneas de detalle. Es aquí donde algunos artículos exponen que las bases de datos NoSQL carecen de confidencialidad e integridad, para salvaguardar la integridad, esto dado a que la naturaleza heterogénea de las bases de datos NoSQL dificulta la protección de la integridad de los datos. [3]. Con lo cual se encuentra una carencia del sistema que se investigará cómo resolver, sin embargo, es uno de los puntos a tener en cuenta al analizar las tecnologías y decidir qué tan aceptables son las problemáticas que puedan presentar.

3. **Escalabilidad y disponibilidad:** Los picos de tráfico durante eventos de alta demanda requieren una infraestructura con tolerancia a fallos y capacidad de escalamiento horizontal. En este sentido, se analiza la gestión de datos en entornos de computación en la nube y las características de los sistemas NoSQL y NewSQL. Entre las estrategias utilizadas para distribuir la carga y mejorar la disponibilidad se encuentran el *sharding* y la replicación, que permiten distribuir los datos entre varios nodos y mantener copias de estos, respectivamente [2]. Estas características hacen que determinadas arquitecturas NoSQL sean una alternativa para aplicaciones que necesitan adaptarse a variaciones importantes en la demanda.

Ante este escenario, se vuelve indispensable migrar hacia un paradigma no relacional (NoSQL) orientado a documentos en la nube, el cual permita flexibilizar el almacenamiento del catálogo, garantizar la persistencia histórica e inmutable de las transacciones y asegurar un rendimiento analítico avanzado y escalable.

B. Objetivos y alcance

Para el objetivo general se busca diseñar e implementar un modelo de base de datos no relacional orientado a documentos utilizando MongoDB Atlas para la plataforma de comercio electrónico MercadoTico 360, con el fin de optimizar la gestión de inventarios heterogéneos, asegurar la inmutabilidad del historial de pedidos y garantizar la escalabilidad horizontal del sistema. Seguidamente para los objetivos específicos se tienen:

1. **Configurar** la arquitectura de almacenamiento en la nube a través de clústeres distribuidos en MongoDB Atlas para asegurar la alta disponibilidad, el aislamiento de consultas y la tolerancia a fallos de la plataforma.

2. **Evaluar** las limitantes del modelo relacional frente a las ventajas del modelo NoSQL orientado a documentos para entornos transaccionales de alta velocidad y volumen.

3. **Estructurar** un esquema de datos flexible que consolide en una única colección un catálogo de productos heterogéneo (alimentos, ropa, tecnología, artesanías y servicios) sin incurrir en redundancias ni campos nulos.

4. **Desarrollar** un patrón de documentos para la colección de pedidos que encapsule las líneas de detalle de las compras, resguardando la integridad histórica de los precios y cantidades facturadas.

El presente proyecto comprende el diseño conceptual, lógico y físico del modelo de datos de MercadoTico 360, abarcando de forma exclusiva cuatro colecciones nucleares: Productos, Clientes (con segmentación geográfica costarricense por provincia, cantón y distrito), Pedidos (historial transaccional) y Comercios.

El alcance técnico se limita al modelado de datos dentro del ecosistema de MongoDB Atlas y la definición de estrategias de agregación analítica de ventas. Queda fuera del alcance de esta investigación el

desarrollo de las interfaces de usuario, la lógica de negocio de la aplicación externa a la base de datos, y los sistemas de pasarelas de pago o logística de distribución física.

II. FUNDAMENTOS TEÓRICOS

A. Fundamentos del modelo NoSQL

Las bases de datos NoSQL nacen como una alternativa para almacenar y consultar datos sin tener que amarrarse a la estructura encasillada de tablas y columnas que usan las bases de datos relacionales tradicionales. En lugar de eso, modelos como el orientado a documentos permiten guardar la información en formatos tipo JSON o BSON. Esta arquitectura facilita escalar rápidamente cuando se manejan grandes volúmenes de datos que no están del todo estructurados.

Además, al distribuir la información entre varios servidores (tanto locales como en la nube), el sistema gana mucha tolerancia a fallos. Si algún servidor o nodo llega a fallar o desconectarse, la base de datos puede seguir funcionando sin interrumpir el servicio. En resumen, adoptar un modelo NoSQL le permite a las empresas actuales procesar muchísimos datos a alta velocidad y adaptarse fácil al crecimiento de sus aplicaciones [4].

Por su parte, la base de datos orientada a documentos es una representación del concepto clave-valor, pero organizada en documentos con campos que se pueden indexar para estructurar la información de forma jerárquica. Está diseñada para guardar sesiones generales, archivos, perfiles de usuario o páginas web [5].

B. Tecnología seleccionada: MongoDB

Para atender los requerimientos operacionales y analíticos del caso de negocio MercadoTico 360, se seleccionó el sistema de gestión de bases de datos orientado a documentos MongoDB.

La elección de esta tecnología se fundamenta en las siguientes características clave de su modelo de datos:

1. **Esquema flexible para un catálogo heterogéneo:** Permite almacenar productos de diversas categorías (ropa, alimentos, tecnología) dentro de una misma colección, gestionando atributos dinámicos sin requerir campos nulos ni alterar la estructura global de la base de datos.
2. **Documentos embebidos para los pedidos:** Permite anidar arreglos como las líneas de detalle dentro de los documentos de pedidos. Esto asegura la inmutabilidad e integridad del historial de compras frente a cambios futuros en el catálogo, además de optimizar las lecturas al evitar la ejecución de operaciones de unión (JOINs) costosas en tiempo de ejecución.
3. **Escalabilidad y evolución sin interrupciones:** Facilita la incorporación de nuevos atributos en tiempo real a medida que la plataforma expande sus líneas de negocio.
4. **Capacidad de análisis avanzado:** MongoDB incorpora un potente motor de agregaciones, ideal para transformar datos operacionales en métricas analíticas sobre el rendimiento de las ventas y clientes.

La elección de estas estrategias de modelado (esquemas flexibles y documentos embebidos) se alinean directamente con las mejores prácticas recomendadas en la documentación oficial de MongoDB para el diseño de bases de datos orientadas a documentos [6].

III. DISEÑO Y METODOLOGÍA

A. Modelo de datos y arquitectura

La arquitectura del sistema adopta un enfoque orientado a la nube centrado en MongoDB Atlas como motor de almacenamiento, lo que permite gestionar la escalabilidad horizontal mediante un clúster distribuido. Esta infraestructura separa la capa de almacenamiento de los clientes o servicios de consulta, garantizando alta disponibilidad, así como alta tolerancia a fallos.

El diseño del esquema de datos se compone de cuatro colecciones principales estratégicamente estructuradas para optimizar el rendimiento y la consistencia:

1. **Colección de productos (Catálogo heterogéneo):** Utiliza un patrón de esquema flexible que permite almacenar ítems de categorías variadas (alimentos, ropa, tecnología, artesanías y servicios) en una misma colección, incluyendo atributos personalizados según el tipo de producto dentro de un subdocumento embebido (atributos), evitando campos vacíos o nulos.

2. **Colección de comercios: 1.** Representa los establecimientos afiliados a la plataforma. Se implementa mediante documentos independientes referenciados desde otras colecciones a través de una clave única (_id). Cada documento almacena la información general del comercio (nombre, correo, teléfono, categoría y estado operativo) y utiliza un objeto embebido para gestionar su dirección física (provincia, cantón, distrito y dirección exacta)

3. **Colección de clientes:**
Corresponde a los datos de los usuarios de la plataforma. Se gestiona mediante documentos independientes referenciados desde otras colecciones a través de una clave única (cliente_id). Incluye información de contacto y una estructura embebida para la dirección física (provincia, cantón, distrito).

4. **Colección de pedidos (Historial transaccional):** Implemente el patrón de documentos embebidos, en lugar de normalizar los detalles de la compra en tablas hijas, cada documento de pedido contiene un arreglo (líneas_detalle) con la lista de productos adquiridos, sus cantidades y precios al momento de la transacción. Este diseño asegura la inmutabilidad histórica del pedido y elimina la necesidad de realizar operaciones JOIN complejas cuando se realiza la lectura del código.

B. Implementación

La lógica de acceso a datos del MercadoTico 360 se implementó en Python utilizando Pymongo como interfaz de comunicación con MongoDB. Las operaciones se organizaron según los principales casos de uso del sistema, como la búsqueda flexible de productos, consulta y actualización de pedidos y obtención de información agregada para apoyar el análisis del comportamiento comercial.

Una necesidad clave de la plataforma es permitir la búsqueda de productos mediante distintos criterios. Dado que los productos poseen categorías distintas con atributos que pueden variar, se implementó una consulta dinámica sobre los atributos utilizando la notación de campos anidados de MongoDB.

```
def buscar_por_atributo(atributo, valor):
    return list(productos.find({
      f"atributos.{atributo}": valor
    }))
```

Listado 1. Consulta de productos por atributo variable.

Como se observa en el Listado 1, el nombre del atributo se incorpora de forma dinámica en la consulta. Esta operación permite buscar, por ejemplo, una talla cuando la categoría del producto es Ropa o el peso en gramos cuando la categoría del producto es Alimentos, sin imponer un mismo conjunto de atributos a todos los productos.

```
productos.find({
    "categoria": categoria,
    "precio": {
        "$gte": precio_minimo,
        "$lte": precio_maximo
    }
})
```

Listado 2. Consulta combinada por categoría y rango de precio.

Adicionalmente, como se visualiza en el Listado 2, se implementaron consultas combinadas por categoría y rango de precio a través de operadores $gte y $lte. Así, una búsqueda puede restringirse tanto por tipo de producto como por su precio simultáneamente, lo cual corresponde a un caso común dentro de un mercado como el del caso estudiado.

La gestión de pedidos es también otra operación central del caso. Cada documento de un pedido mantiene una referencia al cliente que lo solicitó mediante cliente_id y conserva de forma embebida los detalles del encargo. Esto permite recuperar el historial completo de un cliente mediante un filtro directo y reconstruir cada compra con los datos históricos cadados.

```
def buscar_pedidos_por_cliente(cliente_id):
    return list(
        pedidos.find({
            "cliente_id": cliente_id
        })
    )
```

Listado 3. Consulta de pedidos completos por cliente.

El Listado 3 muestra que, para recuperar el historial de un cliente, se filtran los documentos de pedidos mediante el identificador cliente_id. Así, dado que las líneas de detalle de cada pedido se encuentran embebidas, el resultado contiene la información necesaria para reconstruir cada compra.

```
pedidos.update_one(
    {"_id": id_pedido},
    {"$set": {"estado": nuevo_estado}}
)
```

Listado 4. Actualización del estado de un pedido mediante $set.

Para la actualización del estado de un pedido, en el Listado 4 se muestra que se utiliza el operador $set, logrando así evitar reemplazar el documento completo cuando solo cambia su estado. Se mantienen intactos

detalles como la información del cliente, el monto total y las líneas de detalle.

Con el fin de obtener información consolidada se desarrollaron pipelines de agregación. Una de las operaciones más relevantes calcula las ventas por categoría de producto. El pipeline excluye inicialmente a los pedidos cancelados, separa las líneas de detalle de los pedidos y posteriormente relaciona cada producto con su documento correspondiente.

```
pipeline = [
    {"$match": {
        "estado": {"$ne": "Cancelado"}
    }},
    {"$unwind": "$lineas_detalle"},
    {"$lookup": {
     "from": "productos",
     "localField": "lineas_detalle.producto_id",
     "foreignField": "_id",
     "as": "producto"
    }},
    {"$unwind": "$producto"},
    {"$group": {
        "_id": "$producto.categoria",
        "ventas_totales": {
            "$sum": "$lineas_detalle.subtotal"
        }
    }}
]
```

Listado 5. Agregación de ventas por categoría.

En el Listado 5, se observa como este pipeline utiliza funciones como $match, $unwind, $lookup y $group para realizar las acciones descritas anteriormente. Así, se logran acumular las ventas totales por cada categoría.

Una segunda agregación implementada permite identificar los productos con mayor número de ventas. Para ello, nuevamente se excluyen los pedidos cancelados, se separan las líneas de detalle y se agrupa por el id del producto.

```
pipeline = [
    {"$match": {
        "estado": {"$ne": "Cancelado"}
    }},
    {"$unwind": "$lineas_detalle"},
    {"$group": {
        "_id": "$lineas_detalle.producto_id",
        "nombre_producto": {
             "$first":
"$lineas_detalle.nombre_producto"
         },
         "unidades_vendidas": {
            "$sum": "$lineas_detalle.cantidad"
        }
    }},
    {"$sort": {
        "unidades_vendidas": -1
    }},
    {"$limit": 10}
]
```

Listado 6. Agregación de productos más vendidos.

Como se observa en el Listado 6, inicialmente se acumulan las cantidades vendidas para cada producto y luego se ordenan de mayor a menor cantidad de unidades vendidas. Se restringe para que solo se vean los primeros 10 productos. Además de estas operaciones, se incluyeron otras consultas de apoyo, como la agregación de cantidad de pedidos por estado o fecha y la búsqueda de pedidos con un producto específico.

IV. PRUEBAS Y RESULTADOS

A. Entorno de pruebas

Las pruebas se ejecutaron sobre una instancia única de MongoDB 7.0.28 desplegada en local, con Python 3.15.0 y la biblioteca PyMongo 4.18.3 (Tabla I). Se utilizó una instancia local, y no el clúster compartido en Atlas, para que las mediciones sean reproducibles sin credenciales y porque las pruebas crean y eliminan índices.

La base contiene 500 comercios, 10 000 clientes, 25 000 productos en 8 categorías y 50 000 pedidos de 2 a 5 líneas de detalle cada uno. Los datos son sintéticos y se generaron con la biblioteca Faker con semilla fija (Faker.seed(123) y random.seed(123)); los conjuntos finales están versionados en el repositorio. Se verificó que regenerar cada archivo produce el mismo contenido.

Las pruebas se realizaron con el fin de comprobar el desempeño, así mismo se realizaron las pruebas funcionales y las de rendimiento. Las pruebas funcionales permitieron la verificación de las operaciones de inserción, consulta, modificación y validación de documentos. Así mismo, las pruebas de rendimiento involucraron las comparaciones de consultas con índices y sin índices, agregaciones, carga masiva y situaciones de concurrencia.

En las mediciones se utilizaron scripts de Python, registrando sus tiempos de ejecución, latencias y el número de documentos analizados. Los resultados se guardaron en archivos CSV y JSON, facilitando el análisis de la base de datos y la creación de gráficos comparativos. Esto permitió analizar el comportamiento de la base de datos.

TABLA I
ENTORNO DE PRUEBAS

Componente	Especificación
Sistema operativo	Windows 11 (10.0.26200)
Procesador	Intel, 12 núcleos lógicos
Memoria RAM	15.7 GB
Almacenamiento	SSD NVMe, 512 GB
MongoDB	7.0.28 (instancia única, local)
Python / PyMongo	3.15.0 / 4.18.3

Cada prueba de rendimiento se ejecutó una vez como calentamiento, que se descartó, y luego 30 veces (5 en las agregaciones), y se reportan la mediana y el percentil 95 de los tiempos. La medición sin índice se hizo primero, y los índices se crearon después sobre los mismos datos. Además del tiempo, se registraron los documentos examinados y la etapa del plan de ejecución con explain("executionStats"). El código (Performance/scripts/benchmark.py) y los resultados crudos están en el repositorio.

B. Resultados

Efecto de los índices. Sin índices, las siete consultas evaluadas recorrieron la colección completa (*COLLSCAN*: 25 000 documentos en *productos* y 50 000 en *pedidos*). Con los seis índices definidos usaron *IXSCAN* y examinaron entre el 0.01 % y el 13 % de la colección (Tabla II, Fig. 1 y Fig. 2). Las consultas más selectivas mejoraron más: la búsqueda de pedidos de un cliente pasó de 114.9 a 0.5 ms (unas 211 veces más rápida) y la de pedidos que contienen un producto, de 234.8 a 0.5 ms (unas 477 veces). La búsqueda por categoría y rango de precio (Q3) mejoró un 90 % y la que añade la talla (Q4) un 88 %. En Q4 el optimizador eligió el índice sobre la talla (*idx_atributos_talla*) y examinó 1 273 documentos; un índice compuesto que incluya la talla podría reducir esa cifra, algo que no se evaluó.

La consulta solo por categoría (Q1) mejoró apenas un 22 % (de 39.3 a 30.5 ms), aunque redujo los documentos examinados de 25 000 a 3 270. Devuelve cerca de la octava parte del catálogo, por lo que probablemente el costo se concentra en leer y transferir esos documentos y no en localizarlos; es una hipótesis que no se contrastó. En general, un índice aporta menos cuando la consulta devuelve una fracción grande de la colección. La construcción de los índices tomó entre 0.2 y 0.6 s, salvo el de *lineas_detalle.producto_id* (1.4 s), por ser multikey sobre un arreglo.

TABLA II
CONSULTAS SIN Y CON ÍNDICE (30 REPETICIONES)

Consulta	Mediana (ms) sin / con índice	p95 (ms) sin / con índice	Docs. examinados sin / con
Q1 Categoría	39.3 / 30.5	64.6 / 40.8	25 000 / 3 270
Q2 Rango de precio	62.4 / 10.4	75.1 / 14.8	25 000 / 990
Q3 Categoría + precio	47.1 / 4.9	61.7 / 7.3	25 000 / 329
Q4 Categoría + precio + talla	51.7 / 6.1	68.3 / 10.8	25 000 / 1 273
Q5 Productos de un comercio	48.0 / 1.0	56.1 / 1.8	25 000 / 54
Q6 Pedidos de un cliente	114.9 / 0.5	125.8 / 1.1	50 000 / 7
Q7 Pedidos que contienen un producto	234.8 / 0.5	244.8 / 0.9	50 000 / 5

Mediana y percentil 95 en milisegundos; documentos examinados por consulta.

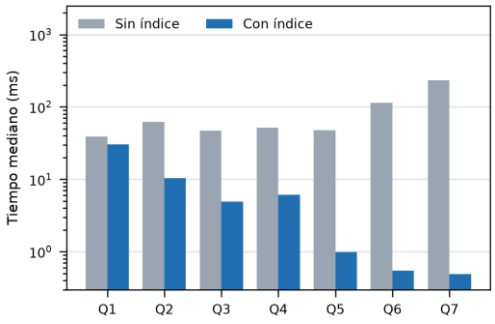


Fig. 1. Tiempo mediano por consulta, sin y con índice (escala logarítmica).

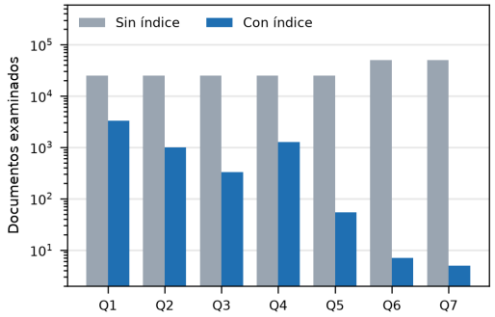


Fig. 2. Documentos examinados por consulta, sin índice y con índice.

Agregaciones. El ticket promedio se calculó en 259 ms y los productos más vendidos en 757 ms (mediana de cinco repeticiones). Las ventas por categoría tardaron 38.6 s (percentil 95: 39.6 s), porque la línea del pedido no guarda la categoría y requiere un *$lookup* a *productos* por cada línea (Fig. 3). Es una limitación del diseño actual; la mejora propuesta es guardar la categoría dentro de *lineas_detalle*.

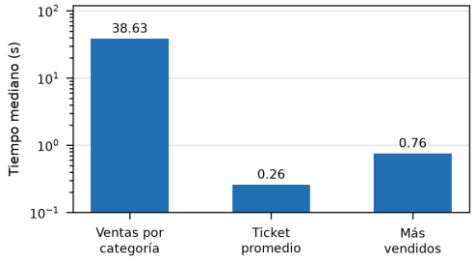


Fig. 3. Tiempo mediano de las agregaciones (escala logarítmica).

Concurrencia. Con 10, 50 y 100 hilos (20 solicitudes por hilo, con índices), el sistema sostuvo entre 560 y 621 solicitudes por segundo. La latencia media subió de 14.4 a 76.3 ms y el percentil 95 de 33.2 a 196.6 ms (Tabla III, Fig. 4): el rendimiento se estabiliza desde los 10 hilos y lo que crece es el tiempo de espera. La prueba usa un solo proceso de Python, cuyo bloqueo global del intérprete limita el paralelismo.

TABLA III
LECTURAS CONCURRENTES (CON ÍNDICES)

Hilos	Solicitudes/s	Latencia media (ms)	p95 (ms)
10	611.7	14.4	33.2
50	621.4	44.3	105.8
100	559.5	76.3	196.6

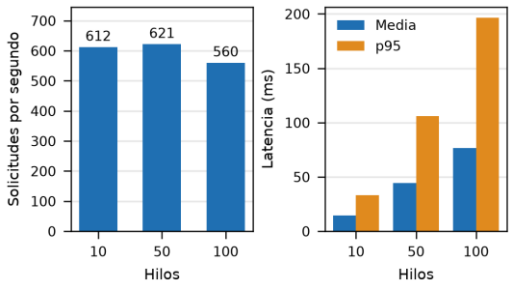


Fig. 4. Rendimiento y latencia con 10, 50 y 100 hilos concurrentes.

Carga masiva. Insertar los 85 500 documentos tomó unos 2.1 s (alrededor de 40 000 documentos por segundo; 31 251 por segundo en los pedidos), sin contar la lectura de los archivos JSON.

Verificación funcional. Se superaron 14 de 15 pruebas funcionales. La omitida (V5) comprueba que MongoDB rechace un producto inválido y no pudo ejecutarse, porque la colección *productos* no tiene validación *$jsonSchema*: la validación existe solo en la capa de Python. Las pruebas superadas cubren los atributos propios por categoría sin campos nulos, la búsqueda por categoría, precio y atributo, la recuperación de los pedidos de un cliente, la actualización del estado (que modifica solo ese campo), la conservación del precio original en el historial cuando cambia el producto, las agregaciones iguales a un cálculo manual, la evolución del esquema sin migración y el rechazo de datos inválidos por las funciones del modelo.

Corridas preliminares. Tres corridas previas, sobre datos generados sin semilla fija, mostraron el mismo patrón en seis de las siete consultas. En Q1 el resultado varió entre un empeoramiento del 40 % y una mejora del 37 %, por lo que el efecto de ese índice sobre esa consulta se considera pequeño e inconstante.

V. DISCUSIÓN Y LIMITACIONES

A. Hallazgos y riesgos observados

Los resultados muestran que el modelo documental cubre los requisitos del caso, pero también dejaron en evidencia riesgos que conviene documentar.

Eficacia de los índices según la selectividad. Los índices redujeron entre un 83 % y un 99.8 % el tiempo mediano de seis de las siete consultas evaluadas, pero solo un 22 % el de la consulta por categoría, que devuelve cerca de la octava parte del catálogo. Esto sugiere definir los índices a partir de los patrones de consulta reales y no indexar por defecto.

Costo del *$lookup* en el diseño de pedidos. Las ventas por categoría tardaron unos 38.6 s porque la categoría no se guarda en las líneas de pedido y hay que consultarla en *productos* por cada línea. Embeber las líneas conserva el historial y evita uniones al leer un pedido, pero traslada el costo a ciertos reportes. Como mejora, se propone copiar la categoría dentro de cada línea.

Validación en la capa de aplicación. Las reglas de validación (precios no negativos, categorías y estados permitidos, subtotales coherentes) existen solo en las funciones de Python. La colección *productos* no tiene un validador *$jsonSchema*, por lo que un documento insertado directamente en MongoDB no se rechaza; esa prueba (V5) quedó omitida. Las referencias entre colecciones (*cliente_id*, *comercio_id*, *producto_id*) son lógicas y no equivalen a llaves foráneas. Esto coincide con la advertencia de [3] sobre la integridad en sistemas NoSQL.

Reproducibilidad de los datos. Durante el desarrollo se detectó que fijar la semilla de Faker no fija el módulo *random* de Python, por lo que los generadores producían datos distintos en cada ejecución. Se corrigió con *random.seed(123)* y un rango fijo de fechas, y se

verificó que cada archivo se regenera con el mismo contenido. Las corridas anteriores a esa corrección se consideran exploratorias.

Variabilidad de las mediciones. El rendimiento en concurrencia varió entre corridas (de 305 a 441 solicitudes por segundo en las preliminares y de 560 a 621 en la final), y el efecto del índice sobre la consulta por categoría cambió de signo entre ellas. No se midió la causa; se atribuye a la carga del equipo, y por eso se reportan medianas y percentiles de varias repeticiones, y la corrida final se hizo con el equipo sin otras tareas.

B. Limitaciones del trabajo

- **Instancia única local.** No se evaluaron la replicación, la conmutación por error ni el particionado (*sharding*). Las propiedades de alta disponibilidad y escalabilidad horizontal descritas en la literatura [2] no se midieron, y tampoco el rendimiento en MongoDB Atlas (latencia de red y recursos compartidos).
- **Cliente de prueba.** La concurrencia se simuló con hilos de un solo proceso de Python, cuyo bloqueo global del intérprete limita el paralelismo real.
- **Escala.** Los 85 500 documentos caben en memoria; no se midió con caché fría ni con volúmenes mayores, donde el efecto de los índices podría ser distinto.
- **Datos sintéticos.** Las distribuciones de categorías, precios y atributos son uniformes y los nombres de producto son palabras aleatorias, por lo que pueden no reflejar un marketplace real.
- **Alcance de las pruebas funcionales.** Se ejecutan sobre un conjunto mínimo de documentos y una muestra de 1 000 pedidos, y comprueban la validación de Python, no la de MongoDB.
- **Índices evaluados.** Se probaron seis índices definidos de antemano; no se exploraron alternativas, como un índice compuesto que incluya la talla.
- **Alcance del proyecto.** No se desarrolló una interfaz de usuario, ni pasarela de pagos ni sistema logístico.

VI. CONCLUSIONES

A. Conclusiones

El modelo implementado en MongoDB permitió representar el catálogo heterogéneo de MercadoTico 360 en una sola colección, que contiene los atributos propios por categoría y sin campos nulos, y conservar en cada pedido las líneas de detalle con el nombre y el precio del momento de la compra.

Las pruebas funcionales han confirmado estos comportamientos (siendo 14 de 15 superadas), en la cual se incluyó la evolución del esquema sin migración y la conservación del precio original cuando se cambia el producto.

Los índices fueron determinantes para el desempeño del proyecto. Donde abarcaba los seis índices definidos, seis de las siete consultas evaluadas redujeron el tiempo mediano entre un 83 % y un 99.8 %. Por ejemplo: la recuperación de los pedidos de un cliente pasó de 114.9 a 0.5 ms. Así mismo la consulta solo por categoría mejoró un 22 %, donde se muestra que el beneficio depende de la selectividad de la consulta.

La evaluación también dejó limitaciones concretas: como las ventas por categoría que tardaron unos 38.6 s por el $lookup el cual lo exige el diseño actual. La validación de datos que existe solo en la capa de Python, sin $jsonSchema en MongoDB; y la latencia aumentó con la concurrencia (siendo de 14.4 a 76.3 ms de media entre 10 y 100 hilos). Las mediciones se hicieron sobre una instancia única local, por lo tanto no se evaluó la replicación, el particionado y el rendimiento en Atlas.

En conclusión, los resultados respaldan el uso de un modelo documental utilizando una indexación basada en patrones de consulta ya que el proyecto lo emérita, siempre que se atiendan las limitaciones señaladas.

B. Trabajo futuro

Como trabajo futuro se propone:

1. Guardar la categoría dentro de las líneas de detalle para evitar el $lookup en las ventas por categoría.
2. Agregar validación $jsonSchema en MongoDB.
3. Evaluar índices compuestos que incluyan atributos variables, como la talla.
4. Repetir las pruebas en un clúster de MongoDB Atlas, con réplicas y particionado, con mayores volúmenes de datos y con un cliente de carga multiproceso.
5. Desarrollar una interfaz que permita gestionar comercios, productos, clientes y pedidos.

Para trabajos futuros se debería incluir pruebas con los conjuntos de datos más amplios y agregar

configuraciones de estructura distintas, así se podría analizar el comportamiento y la solución propuesta en situaciones de crecimiento. Además, se podría crear una aplicación que contenga una interfaz gráfica que facilite la interacción que se llegara a tener con la base de datos, logrando gestionar los comercios, productos, clientes y pedidos. Finalmente, como última sugerencia, se plantea mejorar los mecanismos de validación, seguridad y seguimiento para llevar a cabo la solución a un estado óptimo para su uso en la producción.

***Disponibilidad del código*:** El código, scripts de implementación y datasets utilizados en este trabajo están disponibles públicamente en: https://github.com/emilio667/MercadoTico-360/tree/main?tab=readme-ov-file

REFERENCIAS

[1] S. R. S. A. Maamari and M. Nasar, "A Comparative Analysis of NoSQL and SQL Databases: Performance, Consistency, and Suitability for Modern Applications with a Focus on IoT," *East Journal of Computer Science*, vol. 1, no. 2, pp. 10–15, 2025, doi: 10.63496/ejcs.vol1.iss2.76.

[2] K. Grolinger, W. A. Higashino, A. Tiwari, and M. A. Capretz, "Data management in cloud environments: NoSQL and NewSQL data stores," *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 2, no. 1, 2013, doi: 10.1186/2192-113x-2-22.

[3] N. Gupta and R. Agrawal, "NoSQL Security," in *Advances in Computers*, 2018, pp. 101–132, doi: 10.1016/bs.adcom.2018.01.003.

[4] IBM, "¿Qué son las bases de datos NoSQL?," *IBM Think*, 18 de marzo de 2024. [En línea]. Disponible en: https://www.ibm.com/mx-es/think/topics/nosql-databases. [Accedido: 26-sep-2026]

[5] A. El Alami, Y. Khourdifi, Z. Ait El Mouden, M. Lahmer y M. L. Hasnaoui, "Migrating Relational Databases to NoSQL-Oriented Documents Using Object-Oriented Concepts," *International Journal of Intelligent Engineering and Systems*, *vol. 17*, no. 4, pp. 540-551, 2024, doi: 10.22266/ijies2024.0831.48.

[6] MongoDB, "Data Modeling Introduction," *MongoDB Documentation*, s. f. [En línea]. Disponible en: https://www.mongodb.com/es/docs/manual/data-modeling/. [Accedido: 26-sep-2026].